



Fundamentos de Gestión de Datos

Manual de SQL con SQLite

Bases de datos relacionales desde cero

Entorno: Google Colab · Motor: SQLite

Docente: Pilar Rocío Sayán Mejía

01 ¿Qué es SQL? Historia y comparativo de motores

SQL (Structured Query Language) es el lenguaje estándar para comunicarse con bases de datos relacionales. Fue creado en IBM en 1974 por Donald Chamberlin y Raymond Boyce, basado en la teoría relacional de Edgar Codd. En 1986, ANSI lo adoptó como estándar internacional.

¿Qué hace SQL?

- Crear tablas, insertar datos, consultar información, actualizarla y eliminarla.
- Las 4 operaciones fundamentales: CRUD (Create, Read, Update, Delete).
- SQL no es un lenguaje de programación completo: no tiene bucles ni funciones como Python.

Comparativo de motores de bases de datos relacionales

Motor	¿Quién lo usa?	¿Para qué?	¿Es gratuito?
SQLite	Apps móviles, Colab, cursos	Archivos locales, sin servidor	Sí, 100%
MySQL	WordPress, apps web	Servidores web tráfico medio	Sí (comunidad)
PostgreSQL	Startups, ciencia de datos	Alto rendimiento, datos complejos	Sí, 100%
SQL Server	Empresas grandes, bancos	Sistemas corporativos, reportes	No (Microsoft)
Oracle DB	Grandes corporaciones	Misiones críticas, alto volumen	No (Oracle)

¿Por qué SQLite para este curso?

- SQLite guarda toda la base de datos en un solo archivo .db.
- No necesita instalar ni configurar un servidor.
- Google Colab lo incluye de fábrica.
- Es el motor más usado en aplicaciones móviles (Android, iOS).

02 SQLite en Google Colab: entorno y primer paso

Abre Google Colab en colab.research.google.com, crea una nueva libreta, y en la primera celda escribe lo siguiente:

Celda 1: Importar y conectar (solo se hace UNA vez al inicio)

```
import sqlite3          -- Importa el módulo SQLite que viene con Python/Colab
from tabulate import tabulate -- Para mostrar resultados como tabla bonita

# Crear (o abrir) la base de datos
conn = sqlite3.connect('aynitech.db') -- Crea el archivo aynitech.db en Colab
cur = conn.cursor()      -- El cursor es el 'lápiz' con el que escribimos SQL
print('Conexion exitosa') -- Si no hay error, todo está bien
```

¿Qué es conn y qué es cur?

- conn (connection) = la conexión a la base de datos. Es como abrir la puerta del archivo.
- cur (cursor) = el objeto que ejecuta los comandos SQL. Es como el lápiz que escribe dentro.
- Cada vez que abras Colab debes volver a ejecutar esta celda para reconectar.

Celda 2: Función auxiliar para ver resultados (copiar una sola vez)

```
def ver(sql):          -- Definimos una función llamada ver
    cur.execute(sql)    -- Ejecuta el SQL que le pasemos
    filas = cur.fetchall() -- Obtiene todos los resultados
    cols = [d[0] for d in cur.description] -- Nombres de las columnas
    print(tabulate(filas, headers=cols, tablefmt='grid')) -- Tabla bonita
    print(f'{len(filas)} fila(s) encontrada(s)\n') -- Cuántas filas retornó
```

Flujo de trabajo en Colab con SQLite

- Celda de conexión: import sqlite3 + connect (solo al inicio)
- Celda de función ver(): copiar una sola vez
- Una celda por cada operación SQL: CREATE TABLE, INSERT, SELECT...
- Al terminar: conn.commit() para guardar cambios permanentes

03 Nuestra base de datos ficticia: AyniTech Cusco

A lo largo de este manual construiremos la base de datos de una empresa ficticia llamada AyniTech, una empresa de tecnología ubicada en Cusco, Perú. Tendremos 4 tablas relacionadas entre sí:

Tabla	¿Qué guarda?	Filas de ejemplo
departamentos	Las áreas de la empresa (Sistemas, Ventas...)	5 departamentos
empleados	Los trabajadores y su departamento	8 empleados
proyectos	Proyectos asignados a cada empleado	6 proyectos
ventas	Ventas registradas por cada empleado	10 ventas

Diagrama de relaciones entre tablas

departamentos <--- empleados *(cada empleado pertenece a un departamento)*
empleados <--- proyectos *(cada proyecto es asignado a un empleado)*
empleados <--- ventas *(cada venta la registra un empleado)*

La flecha indica: la tabla de la derecha tiene una FOREIGN KEY que apunta a la tabla de la izquierda.

¿Por qué relacionar las tablas?

- Si guardas el nombre del departamento en cada fila de empleados y lo cambias, tendrías que actualizar cientos de filas. Con relaciones, solo cambias UNA fila.
- Las relaciones eliminan duplicación, aseguran consistencia y permiten reportes cruzados.

04 Crear tablas: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL

Antes de guardar cualquier dato, debemos definir la ESTRUCTURA de cada tabla con CREATE TABLE.

Tipos de datos en SQLite

Tipo SQLite	¿Qué guarda?	Ejemplo
INTEGER	Números enteros	1, 42, -5
REAL	Números decimales	3.14, 1500.50
TEXT	Cadenas de texto	'Cusco', 'Maria Lopez'
NUMERIC	Números exactos (dinero, fechas)	2024-05-10, 1200.00
BLOB	Datos binarios (imágenes, archivos)	Poco usado en cursos

Las 4 restricciones fundamentales

Restricción	¿Qué significa?	Ejemplo
PRIMARY KEY	Identifica cada fila de forma ÚNICA. No puede repetirse ni ser nulo.	id_empleado INTEGER PRIMARY KEY
FOREIGN KEY	Apunta al PRIMARY KEY de otra tabla. Crea la RELACIÓN entre tablas.	id_depto REFERENCES departamentos(id)
UNIQUE	El valor no puede repetirse en la columna, pero SÍ puede ser nulo.	email TEXT UNIQUE
NOT NULL	El campo es OBLIGATORIO: no se puede dejar vacío.	nombre TEXT NOT NULL

Celda 3: Crear la tabla 'departamentos'

```
CREATE TABLE IF NOT EXISTS departamentos ( -- IF NOT EXISTS evita error si ya existe
  id_depto INTEGER PRIMARY KEY AUTOINCREMENT, -- PK: número único, se incrementa solo
  nombre TEXT NOT NULL UNIQUE, -- Obligatorio y no repetible
```

```
ubicacion TEXT NOT NULL          -- Ciudad o piso donde está el área
);
```

Celda 4: Crear la tabla 'empleados'

```
CREATE TABLE IF NOT EXISTS empleados (
  id_emp  INTEGER PRIMARY KEY AUTOINCREMENT, -- PK: identificador único
  de empleado
  nombre  TEXT    NOT NULL,                -- Nombre completo, obligatorio
  cargo   TEXT    NOT NULL,                -- Analista, Desarrollador, Jefe...
  sueldo  REAL    NOT NULL,                -- Sueldo mensual en soles
  email   TEXT    UNIQUE,                  -- Email único por empleado
  ciudad  TEXT    DEFAULT 'Cusco',         -- Si no se indica, se pone Cusco
  id_depto INTEGER NOT NULL,               -- Columna que REFERENCIA a
  departamentos
  FOREIGN KEY (id_depto)                  -- Declaración de la llave foránea
  REFERENCES departamentos(id_depto)      -- Apunta al PK de
  departamentos
);
```

Celda 5: Crear la tabla 'proyectos'

```
CREATE TABLE IF NOT EXISTS proyectos (
  id_proyecto INTEGER PRIMARY KEY AUTOINCREMENT,
  nombre      TEXT    NOT NULL,
  presupuesto REAL    NOT NULL,
  estado      TEXT    DEFAULT 'En curso', -- En curso, Terminado, Pausado
  id_emp      INTEGER NOT NULL,
  FOREIGN KEY (id_emp) REFERENCES empleados(id_emp)
);
```

Celda 6: Crear la tabla 'ventas'

```
CREATE TABLE IF NOT EXISTS ventas (
  id_venta INTEGER PRIMARY KEY AUTOINCREMENT,
  fecha    TEXT    NOT NULL, -- Formato 'YYYY-MM-DD'
```

```
    monto REAL NOT NULL, -- Monto en soles
    producto TEXT NOT NULL, -- Nombre del producto o servicio
    id_emp INTEGER NOT NULL,
    FOREIGN KEY (id_emp) REFERENCES empleados(id_emp)
);

conn.commit()          -- IMPORTANTE: guarda todos los CREATE TABLE
print('Tablas creadas correctamente')
```

Orden obligatorio para crear tablas con relaciones

- Siempre crea PRIMERO la tabla que no tiene FOREIGN KEY (la tabla 'padre').
- Luego crea las tablas que SÍ tienen FOREIGN KEY (las tablas 'hijas').
- En nuestro caso: 1) departamentos → 2) empleados → 3) proyectos y ventas.

05 Insertar datos con INSERT INTO

Una vez creadas las tablas, las llenamos con datos usando INSERT INTO. Los campos INTEGER PRIMARY KEY AUTOINCREMENT NO se insertan: SQLite los genera solos.

Celda 7: Insertar departamentos

```
INSERT INTO departamentos (nombre, ubicacion) VALUES
('Sistemas',      'Piso 3 - Av. El Sol'),  -- id_depto = 1
('Ventas',        'Piso 1 - Av. El Sol'),  -- id_depto = 2
('Administracion', 'Piso 2 - Av. El Sol'),  -- id_depto = 3
('Soporte',       'Piso 1 - Urb. Ttio'),    -- id_depto = 4
('Proyectos',     'Piso 4 - Av. El Sol');  -- id_depto = 5
```

Celda 8: Insertar empleados

```
INSERT INTO empleados (nombre, cargo, sueldo, email, ciudad, id_depto)
VALUES
('Carlos Quispe', 'Desarrollador', 3500.00, 'cquispe@ayni.pe', 'Cusco', 1),
('Lucia Mamani',  'Analista',      2800.00, 'lmamani@ayni.pe', 'Cusco', 1),
('Pedro Condori', 'Jefe de Ventas', 4200.00, 'pcondori@ayni.pe', 'Cusco', 2),
('Ana Huanca',   'Vendedora',      2200.00, 'ahuanca@ayni.pe', 'Puno', 2),
('Jose Ttito',   'Administrador', 3100.00, 'jttito@ayni.pe', 'Cusco', 3),
('Maria Ccopa',  'Soporte TI',     2600.00, 'mccopa@ayni.pe', 'Cusco', 4),
('Roberto Apaza', 'Desarrollador', 3800.00, 'rapaza@ayni.pe', 'Cusco', 1),
('Sofia Huallpa', 'Lider Proyectos', 4500.00, 'shuallpa@ayni.pe', 'Cusco', 5);
```

Celda 9: Insertar proyectos

```
INSERT INTO proyectos (nombre, presupuesto, estado, id_emp) VALUES
('Sistema ERP Cusco', 85000.00, 'En curso', 1),  -- Carlos Quispe
('App Movil Turismo', 42000.00, 'Terminado', 7),  -- Roberto Apaza
('Portal Web Ventas', 30000.00, 'En curso', 8),  -- Sofia Huallpa
('Migracion Base Datos', 55000.00, 'Pausado', 2), -- Lucia Mamani
('Chatbot Soporte', 18000.00, 'En curso', 6),  -- Maria Ccopa
('Dashboard Gerencial', 25000.00, 'Terminado', 8); -- Sofia Huallpa
```


Celda 9b: Insertar ventas

```
INSERT INTO ventas (fecha, monto, producto, id_emp) VALUES
('2024-01-15', 12500.00, 'Licencia Software', 3), -- Pedro Condori
('2024-02-03', 8200.00, 'Consultoria TI', 3),
('2024-02-20', 15000.00, 'Desarrollo Web', 1), -- Carlos Quispe
('2024-03-10', 5500.00, 'Soporte Anual', 6), -- Maria Ccopa
('2024-03-25', 22000.00, 'ERP Modulo RRHH', 1),
('2024-04-05', 9800.00, 'App Movil', 4), -- Ana Huanca
('2024-04-18', 31000.00, 'Sistema Contable', 3),
('2024-05-02', 4500.00, 'Capacitacion SQL', 2), -- Lucia Mamani
('2024-05-15', 17500.00, 'Plataforma eLearning', 7), -- Roberto Apaza
('2024-06-01', 11200.00, 'Dashboard BI', 8); -- Sofia Huallpa

conn.commit() -- Guarda TODOS los INSERT realizados
print('Datos insertados correctamente')
```

06 Consultas con SELECT y WHERE — filtros y reportes

SELECT es el comando para LEER datos de la base de datos. Con WHERE agregamos condiciones para filtrar solo las filas que nos interesan.

Estructura básica del SELECT

```
SELECT columnas      -- Qué columnas quiero ver (o * para todas)
FROM tabla           -- De qué tabla leo los datos
WHERE condicion      -- Solo filas que cumplan esta condición
ORDER BY columna ASC/DESC -- Ordenar resultados (ASC o DESC)
LIMIT n;             -- Mostrar solo los primeros n resultados
```

Celda 10: Ver todos los empleados

```
ver("""
SELECT *             -- El asterisco trae TODAS las columnas
FROM empleados      -- De la tabla empleados
ORDER BY nombre ASC  -- Ordenados de A a Z
```

```
;"")
```

Celda 11: Filtrar empleados de Cusco con sueldo mayor a 3000

```
ver("
  SELECT nombre, cargo, sueldo, ciudad -- Solo las columnas que interesan
  FROM empleados
  WHERE ciudad = 'Cusco' -- Solo empleados cuya ciudad ES Cusco
  AND sueldo > 3000 -- Y ADEMÁS cuyo sueldo sea mayor a 3000
  ORDER BY sueldo DESC -- De mayor a menor sueldo
;"")
```

Operadores disponibles en WHERE

Operador	Significado	Ejemplo en SQL
=	Igual a	ciudad = 'Cusco'
!= o <>	Diferente de	ciudad != 'Lima'
> / <	Mayor / Menor que	sueldo > 3000
>= / <=	Mayor o igual / Menor o igual	sueldo >= 2500
BETWEEN	Entre dos valores	sueldo BETWEEN 2000 AND 4000
LIKE	Buscar patrón en texto	nombre LIKE 'Mar%'
IN (...)	Dentro de una lista	ciudad IN ('Cusco','Puno')
IS NULL	El campo está vacío	email IS NULL
AND	Ambas condiciones	ciudad='Cusco' AND sueldo>3000
OR	Al menos una condición	ciudad='Cusco' OR ciudad='Puno'
NOT	Niega la condición	NOT ciudad = 'Lima'

Celda 12: Otros filtros útiles

```
-- Empleados cuyo nombre empieza con 'C'
ver("SELECT nombre, cargo FROM empleados WHERE nombre LIKE 'C%';")

-- Empleados de Cusco O de Puno
```

```
ver("SELECT nombre, ciudad FROM empleados WHERE ciudad IN  
( 'Cusco','Puno' );")
```

```
-- Empleados con sueldo entre 2500 y 4000
```

```
ver("SELECT nombre, sueldo FROM empleados WHERE sueldo BETWEEN 2500  
AND 4000;")
```

```
-- Solo los 3 empleados con mayor sueldo
```

```
ver("SELECT nombre, sueldo FROM empleados ORDER BY sueldo DESC LIMIT 3;")
```

07 Relaciones entre tablas: JOIN

La tabla 'empleados' guarda id_depto (número) pero NO el nombre del departamento. Para verlo junto con el empleado, debemos UNIR (JOIN) ambas tablas. JOIN combina filas de dos tablas cuando un valor coincide (generalmente PK con FK).

Estructura del INNER JOIN

```
SELECT t1.columna, t2.columna -- Especificamos de qué tabla viene cada  
columna  
FROM   tabla1 t1             -- tabla1 con alias t1 (nombre corto)  
INNER JOIN tabla2 t2         -- Solo filas que tienen coincidencia en ambas tablas  
    ON t1.id = t2.id_fk      -- ON: la condición de unión (PK = FK)  
WHERE condicion;            -- WHERE opcional: filtros adicionales
```

Celda 13: Ver empleados con su departamento

```
ver("  
    SELECT e.nombre AS Empleado, -- e = alias de empleados  
           e.cargo AS Cargo,  
           e.sueldo AS Sueldo,  
           d.nombre AS Departamento, -- d = alias de departamentos  
           d.ubicacion AS Ubicacion  
    FROM   empleados e          -- Tabla principal: empleados  
    INNER JOIN departamentos d  -- Unimos con departamentos  
        ON e.id_depto = d.id_depto -- Donde los id_depto coincidan
```

```
ORDER BY d.nombre, e.sueldo DESC -- Por departamento, luego por sueldo
;")
```

Celda 14: Reporte de ventas con nombre de empleado

```
ver("
  SELECT v.fecha    AS Fecha,
         e.nombre   AS Vendedor,
         v.producto AS Producto,
         v.monto    AS Monto_Soles
  FROM   ventas v
  INNER JOIN empleados e ON v.id_emp = e.id_emp
  ORDER BY v.monto DESC -- Las ventas de mayor a menor monto
;")
```

Celda 15: JOIN de tres tablas — Empleado, Departamento y Proyecto

```
ver("
  SELECT e.nombre    AS Empleado,
         d.nombre    AS Departamento,
         p.nombre    AS Proyecto,
         p.presupuesto AS Presupuesto,
         p.estado     AS Estado
  FROM   proyectos p
  INNER JOIN empleados e ON p.id_emp = e.id_emp
  INNER JOIN departamentos d ON e.id_depto = d.id_depto
  WHERE  p.estado = 'En curso' -- Solo proyectos activos
  ORDER BY p.presupuesto DESC
;")
```

08 Funciones de resumen: COUNT, SUM, AVG, GROUP BY

Las funciones de agregación permiten calcular resúmenes: contar filas, sumar valores, promediar, encontrar el máximo o mínimo. Se combinan con GROUP BY para agrupar resultados por categoría.

Función	¿Qué calcula?	Ejemplo
COUNT(*)	Cuenta el número de filas	COUNT(*) → 8 empleados
SUM(col)	Suma los valores de una columna	SUM(sueldo) → total planilla
AVG(col)	Promedio de los valores	AVG(sueldo) → sueldo promedio
MAX(col)	El valor más alto	MAX(sueldo) → sueldo máximo
MIN(col)	El valor más bajo	MIN(monto) → venta mínima
GROUP BY	Agrupar filas según una columna	GROUP BY departamento
HAVING	Filtra DESPUÉS de agrupar	HAVING AVG(sueldo) > 3000

Celda 16: Estadísticas generales de empleados

```

ver(
  SELECT COUNT(*) AS Total_Empleados,      -- Cuenta todas las filas
         ROUND(AVG(sueldo),2) AS Sueldo_Prom, -- Promedio redondeado a 2
         MAX(sueldo) AS Sueldo_Maximo,      -- El sueldo más alto
         MIN(sueldo) AS Sueldo_Minimo,      -- El sueldo más bajo
         SUM(sueldo) AS Planilla_Total      -- Total que paga la empresa
  FROM empleados
;
)
```

Celda 17: Resumen por departamento

```

ver(
  SELECT d.nombre      AS Departamento,
         COUNT(e.id_emp) AS Num_Empleados,
         ROUND(AVG(e.sueldo),2) AS Sueldo_Promedio,
         SUM(e.sueldo)   AS Total_Planilla
  FROM   empleados e
  INNER JOIN departamentos d ON e.id_depto = d.id_depto
  GROUP BY d.nombre      -- Agrupa los resultados por departamento
  ORDER BY Total_Planilla DESC
)
```

```
;''')
```

Celda 18: Total de ventas por empleado (solo los que vendieron más de 20000)

```
ver(''  
  SELECT e.nombre      AS Vendedor,  
         COUNT(v.id_venta) AS Num_Ventas,  
         SUM(v.monto)   AS Total_Vendido  
  FROM   ventas v  
  INNER JOIN empleados e ON v.id_emp = e.id_emp  
  GROUP BY e.nombre  
  HAVING SUM(v.monto) > 20000 -- Filtra grupos: solo los que vendieron más  
de 20000  
  ORDER BY Total_Vendido DESC  
;''')
```

09 Resumen de comandos SQL esenciales

Comando	¿Para qué sirve?	Ejemplo básico
CREATE TABLE	Define la estructura de una tabla nueva	CREATE TABLE clientes (...)
INSERT INTO	Agrega filas de datos a una tabla	INSERT INTO t VALUES (...)
SELECT ... FROM	Lee y muestra datos de una o más tablas	SELECT * FROM empleados
WHERE	Filtra filas según una condición	WHERE ciudad = 'Cusco'
ORDER BY	Ordena los resultados ASC o DESC	ORDER BY sueldo DESC
LIMIT n	Muestra solo los primeros n resultados	LIMIT 5
INNER JOIN ... ON	Une dos tablas por una columna común	JOIN dept ON e.id = d.id
GROUP BY	Agrupar filas para aplicar funciones de resumen	GROUP BY departamento
HAVING	Filtra grupos (después de GROUP BY)	HAVING COUNT(*) > 2
COUNT / SUM / AVG	Cuentan, suman o promedian valores	COUNT(*), SUM(monto)
MAX / MIN	Valor más alto o más bajo de una columna	MAX(sueldo)
UPDATE ... SET	Modifica datos existentes en una tabla	UPDATE emp SET sueldo=4000 WHERE id=1
DELETE FROM	Elimina filas de una tabla	DELETE FROM emp WHERE id=1
DROP TABLE	Elimina la tabla completa con todos sus datos	DROP TABLE IF EXISTS emp
conn.commit()	Guarda permanentemente los cambios en SQLite	Siempre al final

Orden correcto de cláusulas en un SELECT

SELECT (qué columnas mostrar)
FROM (de qué tabla o tablas)
JOIN (unir con otras tablas si es necesario)
WHERE (filtrar filas — ANTES de agrupar)
GROUP BY (agrupar resultados)
HAVING (filtrar grupos — DESPUÉS de agrupar)
ORDER BY (ordenar el resultado final)
LIMIT (limitar cantidad de filas)

Manual de SQL con SQLite · TECSUP · Fundamentos de Gestión de Datos

Docente: Pilar Rocío Sayán Mejía